DETECTION ENGINEERING · HOME SOC LAB

Custom SIEM Rule Development & Live Validation

Building, breaking, and verifying detections on Wazuh — and the failure modes that hide in plain sight

---

Analyst	Olutayo Aladeusi
Discipline	Detection Engineering / SOC Analysis
Platform	Wazuh 4.12.0 SIEM — manager, Linux & Windows agents
Target	DVWA (Damn Vulnerable Web Application)
Attacker tooling	OWASP ZAP, curl, Sysmon telemetry
Date	29–30 August 2026

Prepared as a portfolio work sample. All testing was performed against self-hosted, deliberately vulnerable systems owned by the analyst on an isolated lab network. No third-party or production systems were involved.

1. Executive Summary

This engagement authored a 48-rule custom detection ruleset for the Wazuh SIEM, deployed it against a live vulnerable target, and — critically — **validated every rule by firing real attacks through it** rather than trusting that it loaded.

The central finding is one every detection engineer eventually learns the hard way: **"the rule loaded" and "the rule works" are unrelated statements.** A SIEM gives no signal to distinguish them. Every rule in this ruleset passed XML validation and appeared in the loaded ruleset; several detected nothing. Nine distinct failure modes were identified and documented, most producing no error and no visible symptom on the dashboard.

48	9	5	100%
CUSTOM RULES AUTHORED	FAILURE MODES FOUND	DETECTION SURFACES VALIDATED LIVE	SCAN-NOISE SUPPRESSED BY TUNING

Four independent detection surfaces were validated against live traffic: web-log scanning, web-attack payloads (including successful-exploitation detection), realtime file-integrity / webshell detection, and SSH authentication abuse. A full OWASP ZAP scan and an authenticated payload run exercised the rules at volume, and a tuning pass demonstrated scoped suppression of scanner noise from an authorised source.

> **What this demonstrates.** Not "I installed a SIEM," but the actual craft of detection engineering: writing rules, discovering how they silently fail, diagnosing each against live evidence, fixing them, and honestly separating verified detections from unverified ones — including discarding one misleading metric that was a logging artifact rather than a real result.

2. Lab Environment & Architecture

The lab runs entirely on WSL2 distributions plus a Windows host, sharing one network namespace. This is a deliberately modest footprint that still exercises a realistic manager/agent SIEM topology.

Component	Role	Detail
Ubuntu (WSL2)	Wazuh Manager 4.12.0	Rule engine, correlation, alert store; 48 custom rules in `local_rules.xml`
OracleLinux 9.5 (WSL2)	Target — agent **`ol-dvwa-01`** (ID 003)	LAMP + DVWA (security: Low); Apache access log + realtime FIM shipped to manager
Kali (WSL2)	Attacker	OWASP ZAP (Docker), curl payload delivery, nmap
Windows host	Agent **`MyAgentCIA`** (ID 001)	Sysmon process/file telemetry via `windows_eventchannel`

Platform limitations (stated honestly, not worked around)

Limitation	Effect	Rules affected
WSL2 has no audit kernel subsystem	`auditd` command logging unavailable	Linux post-exploitation (100060–100067) — inert by platform
`ufw` logging unreliable under WSL2	Firewall events not consistently delivered	Port-scan detection (100050–100052) — untested
Shared network namespace	All hosts present as one source IP	Source-IP attribution is cosmetic in-lab

These constrain coverage but not the core methodology. Each is recorded as a documented constraint rather than presented as working coverage — an honest scope statement being part of the deliverable.

3. Methodology — The Validation Loop

Every finding in this report was surfaced by one repeatable loop. The discipline it encodes is the single most transferable output of the engagement.

1. **Syntax validation before restart.** `wazuh-analysisd -t` — a malformed rule takes the *entire* manager offline, so it is validated before, never after.
2. **Decoder inspection.** `wazuh-logtest` Phase 2 — read the *actual* field names a decoder produces rather than assuming them. This step alone prevents the most common class of silent failure.
3. **Fired-vs-expected comparison.** Fire a representative event and compare the rule ID that *fired* against the one that *should have*. This is the only step that catches semantic failures.
4. **Live verification for un-testable sources.** Where `logtest` cannot reach (Windows eventchannel), generate real behaviour and verify against `alerts.json`.
5. **Vendor-baseline audit before authoring.** Check the shipped ruleset first — a custom rule duplicating vendor coverage is unreachable from the day it is written.

> **Hard-won ordering:** step 5 belongs first. One pre-existing rule (100500) duplicated a vendor rule and had never fired since the day it was created — invisible until tested.

4. Validated Detection Coverage

Four detection surfaces were confirmed firing against live traffic. Each row below was verified by an observed alert, not by inspection.

4.1 Web-log scanning

Rule	Detection	Status
100012	Web 4xx probe candidate	VERIFIED 23/25 probes; `web-accesslog` decoded
100013	Scanner behaviour (10+ 4xx / 30s)	VERIFIED fired 117× under ZAP scan
100016	Sensitive-path probe	VERIFIED 24× (`/.git` , `/etc` , `/var/www`)
100015	Directory enumeration (15+ distinct)	STARVED see Finding 5

4.2 Web-attack payloads

Rule	Detection	Status
100040	SQL injection	VERIFIED `UNION` fires; boolean SQLi evades (gap characterised)
100042	Cross-site scripting	VERIFIED
100043	Path traversal	VERIFIED 61× across runs
100044	Command injection	VERIFIED
100045	Attack returned HTTP 200 — **successful exploitation**	VERIFIED 5 → 33 after fix; the payoff rule

4.3 File integrity / webshell & 4.4 Authentication

Rule	Detection	Status
100030	Webshell dropped in web root	VERIFIED realtime, T1505.003, full SHA-256 captured
100022	SSH user enumeration / brute force	VERIFIED fired at 6th failed auth, T1110.001

5. Findings

Nine findings, ranked by severity. All share one signature: a rule that loads, validates, and reads as correct while detecting nothing — surfaced only by firing events and checking which rule ID actually won.

HIGH F-1 Syntax error takes the entire manager offline

Availability · rule 100014

A reserved static option (`url`) was used as a dynamic `<field>`. `wazuh-analysisd` refused to start — not a degraded rule but a **total detection outage**, taking every existing rule down with it.

Implication: a single malformed custom rule is a full outage, not a partial one. Rule changes must be validated with `wazuh-analysisd -t` before restart and ideally staged off the production manager.

HIGH F-2 Semantically wrong rules load silently and correlate nothing

Detection gap · rules 100020 / 100022 / 100023

`<same_field>srcip</same_field>` loaded without error and never fired: `srcip` is a *static* field, and `same_field` operates only on *dynamic* ones (the correct option is `<same_source_ip/>`). Before the fix, 10 SSH failures produced only built-in rules; after, 100022 fired correctly at the 6th event. Found only by comparing fired-vs-expected rule IDs — there is no error or dashboard signal.

HIGH F-4 Custom and vendor rules shadow each other

Silent coverage change · bidirectional

Wazuh fires **one rule per event**; when several match, the loser is invisible. Observed both ways: a custom rule (100022) displaced a built-in (5712) that then stopped firing entirely; and a built-in (92057) displaced a pre-existing custom rule (100500) that had **never once fired**. Adding custom rules silently *reduced* existing coverage, with no warning.

HIGH F-6 The payoff rule was broken by a wrong field name

Most valuable rule, silently inert · rule 100045

Rule 100045 — "web attack returned HTTP 200 → likely successful exploitation" — is the point of the whole ruleset: it separates an attack *attempted* from one that *worked*. It checked `<status>`, but the web decoder stores the HTTP code in `<id>`. It could never match.

How it surfaced: only by authenticating a real DVWA session, firing a union SQLi that returned `200`, and noticing the *attempt* rule incremented while the *success* rule did not. Fixed (`status` → `id`) and confirmed firing.

> **Lesson:** the rules most worth having are the hardest to test — they need the target in a specific state (authenticated, vulnerable, returning success). A ruleset that has never had a *successful* attack run through it has never tested its most important rules.

MEDIUM F-3 Test methodology can invalidate results without failing

Process · Windows/Sysmon rules

Synthetic Sysmon JSON decoded perfectly in `logtest` but matched no rule — the Windows rule tree requires the `windows_eventchannel` decoder, which is compiled in and cannot be selected from `logtest`. A negative result produced by a broken harness, not a broken rule; five correct rules would have been wrongly recorded as dead. Correct method: generate real endpoint behaviour and verify against `alerts.json`.

MEDIUM F-5 A low-threshold composite starves its higher-threshold sibling

Detection gap · rules 100013 vs 100015

Both chain off base rule 100012. When 100013 (threshold 10) fires, Wazuh drains the shared match pool, so 100015 (threshold 15) never reaches its count. Across 65 distinct probes, 100013 fired 6× and 100015 **0×** — confirmed structural, not volume-related (still 0 under 1,090 events during the ZAP scan). Resolution: re-scope 100015 to chain off 100013 rather than compete with it.

MEDIUM F-7 "Config didn't load" masquerades as "feature doesn't work"

Diagnostic discipline · rule 100030 (FIM)

A webshell drop didn't trigger FIM, prompting two wrong hypotheses (realtime FIM broken on WSL; multi-block config won't merge). Both false — the agent simply hadn't been restarted after the config change. Once restarted, a fresh webshell fired 100030 in realtime with full hashes. **Lesson:** confirm the config actually loaded before blaming the platform. (Bonus: the first webshell, written via `sudo tee`, was caught anyway by sudo command logging — defense-in-depth by accident.)

INFO F-8 Signature scanner-detection fails; behavioural detection carries it

Design validation · rules 100014 vs 100013

The user-agent scanner rule (100014) fired **0×** during a full ZAP scan — ZAP uses a browser-like UA by default. The same scan fired the *behavioural* rule (100013) **117×**. Behaviour — bursts of 4xx from one source — cannot be masked the way a UA string can. Direct, at-scale evidence for preferring behavioural over signature detection.

INFO F-9 Headless authenticated DAST is not a one-liner

Tooling reality · OWASP ZAP

An authenticated ZAP scan via cookie-injection stayed locked at the login page (20 URLs, same as unauthenticated). Headless authenticated scanning needs a real ZAP *context* with session management and logout exclusion, not a static cookie. The goal — exercising deep payload rules with authenticated 200 responses — was met instead via authenticated curl, which drove the success rule 100045 from 5 to 33.

6. MITRE ATT&CK Coverage

Verified-firing detections mapped to ATT&CK techniques — the framing a SOC or GRC team expects for coverage discussions.

Tactic	Technique	ID	Detection
Reconnaissance	Active Scanning / Wordlist	T1595	100013 web scan burst (117×)
Credential Access	Brute Force: Password Guessing	T1110.001	100022 SSH user enumeration
Initial Access	Exploit Public-Facing Application	T1190	100040/43/44/45 web attacks
Execution	Command & Scripting Interpreter	T1059	100044 OS command injection
Persistence	Server Software Component: Web Shell	T1505.003	100030 realtime webshell FIM
Impact	Inhibit System Recovery (shadow copy)	T1490	100093 (authored; unique vs vendor)

7. Vendor Baseline Audit

Custom Windows rules were audited against the shipped ruleset — the step that should precede authoring, not follow it. Verdicts:

Rule	Intended detection	Vendor coverage	Verdict
100093	Shadow-copy deletion (ransomware)	None found	RETAIN unique
100090	Office/browser spawns shell	`mshta` / `verclsid` only	RETAIN partial gap
100091	LOLBin remote download	`-decode` only	RETAIN `-urlcache` uncovered
100092	LSASS credential dumping	Direct equivalent	SHADOWED annotate
100094	Endpoint security disabled	8 Defender rules	SHADOWED annotate
100500	PowerShell EncodedCommand	Rule 92057 (tighter)	SUPERSEDED

Highest-value custom rule: 100093 — shadow-copy deletion is a top-tier ransomware precursor with no coverage in the shipped ruleset.

8. Tuning & Noise Management

A full ZAP scan generated ~1,300 alerts in minutes, dominated by 100012 (1,090) and 100013 (117) — the raw flood a real scan produces. Rule 100110 was re-scoped to suppress the custom scan rules from an authorised source at level 0 (matched, not written).

Signal (40 identical probes from authorised source)	Result
Probes logged by Apache (traffic confirmed)	**40**
100012 alerts generated	**0**
100013 alerts generated	**0**

Suppression is scoped: only the authorised source and only the custom scan rules are silenced — Wazuh's built-in rules and all other sources still alert normally.

> **Analytical honesty:** a tempting headline metric (total alerts 1,998 → 0) was *discarded* — `alerts.json` had rotated between reads, so the drop was a logging artifact, not tuning. The valid proof is per-probe: 40 requests logged, 0 corresponding custom alerts. Distinguishing evidence from artifact is itself a core analyst skill.

9. SIEM Integrations & Automated Response

Beyond detection rules, the SIEM was extended with four integrations — each proven with a live trigger — turning a detection-only platform into a full **detect → enrich → respond → report** pipeline.

Integration	Capability	Validation
Active response	Auto-quarantine a webshell when rule 100030 fires — file moved to a locked quarantine dir, no human in the loop	VERIFIED shell4.php dropped → detected → quarantined in seconds
VirusTotal	FIM file hashes auto-checked against 70+ AV engines; malicious verdict raises an alert	VERIFIED EICAR flagged 64/67, rule 87105
Suricata IDS	Network-layer detection on eth0; eve.json alerts ingested and correlated by the manager	VERIFIED custom signature → Wazuh rule 86601
Compliance / GRC	Regulatory dashboards (NIST 800-53, PCI DSS, HIPAA, GDPR) + SCA CIS-benchmark scoring, on the full indexer/dashboard stack	LIVE 521 alerts mapped to NIST controls (AU/AC/SI); CIS benchmark scored 47% (71 pass / 80 fail / 179 checks)

> **The response loop, demonstrated.** A webshell written to the web root is detected by realtime FIM (100030), its hash checked against VirusTotal, and the file automatically quarantined — detection, enrichment, and containment with no analyst intervention. Meanwhile every alert is mapped to NIST 800-53 controls for audit evidence. This is the difference between a SIEM that watches and one that acts.

Platform note: Suricata's `af-packet` falls back from fanout mode under WSL (kernel limitation) but captures and alerts correctly; the ICS/SCADA rule content (Modbus, DNP3, ENIP) is installed, with the protocol parsers a one-line enablement for OT-focused work.

10. If This Were Production

1. **Never edit rules on the live manager.** A malformed rule is a total outage (F-1). Stage on a test manager; gate promotion on `wazuh-analysisd -t`.
2. **Treat every new rule as unverified until an event fires it.** Clean startup is not validation. Maintain a test-event corpus per rule.

3. **Audit against the vendor baseline before authoring** (F-4) — and watch for custom rules silently displacing working built-ins.
4. **Prefer behavioural detection over signature** where both exist (F-8): behaviour survives evasion that trivially defeats UA/token signatures.
5. **Prioritise success-detection.** Attempt-alerts are noise at scale; the "attack returned 200" class (F-6) is what warrants escalation — and is the hardest to validate, so validate it deliberately.
6. **Tune with scoped allow-lists, not blanket suppression** (§8): silence the authorised source and the specific noisy rules, nothing broader.

11. Honest Verification Status

A rule that has not fired is *unverified*, not *working*. This distinction is maintained deliberately.

Status	Rules
VERIFIED FIRING	100012, 100013, 100016, 100022, 100030, 100040, 100042, 100043, 100044, 100045
VERIFIED NON-FIRING (shadowed/starved)	100015 (starved), 100500 / 100092 / 100094 (shadowed)
UNVERIFIED — sound logic, untested	100090, 100091, 100093 (100093 destructive to test)
UNVERIFIED — no log source (WSL)	100050–100052 (firewall), 100060–100067 (auditd)

12. Conclusion & Skills Demonstrated

Across two sessions this lab moved from an empty ruleset to a validated, characterised detection pipeline — and, more importantly, to a documented understanding of how detections fail silently and how to catch them.

Capability	Evidence in this report
SIEM administration	Wazuh manager + multi-OS agent deployment, log-source integration, FIM
Detection engineering	48 custom rules; decoder/field analysis; composite correlation; MITRE mapping
Adversary emulation	OWASP ZAP scanning, authenticated payload delivery, webshell drop
Diagnostic rigour	9 failure modes isolated against live evidence; fired-vs-expected method
Analytical integrity	Verified vs unverified maintained; a misleading metric identified and discarded
Alert tuning	Scoped suppression with measured before/after
Automated response (SOAR)	Active-response webshell quarantine; VirusTotal enrichment on FIM
Network detection	Suricata IDS integrated and correlated into the SIEM
GRC / compliance	NIST 800-53 & PCI/HIPAA/GDPR dashboards; CIS-benchmark (SCA) hardening audit

All testing performed on self-hosted, deliberately vulnerable lab systems on an isolated network. No production or third-party systems were accessed. Prepared by Olutayo Aladeusi as a detection-engineering portfolio work sample, August 2026.